

VisionInspect

Automated Visual Defect Inspection using Classical Computer Vision and a From-Scratch Classifier

Field	Detail
Project title	VisionInspect — Automated Visual Defect Inspection
Domain	Computer Vision (with machine-learning evaluation)
Problem addressed	Automatic detection, localisation and classification of surface defects (scratch / spot / crack) from grayscale images
Core techniques	Morphological background estimation, robust MAD thresholding, contour and Hu-moment descriptors, distance-weighted k-NN
Implementation	Python 3, OpenCV, NumPy, Matplotlib
Headline result	100% accuracy and 0 false alarms on a held-out split of 60 images; ≈ 6 ms per image (166 images/s, CPU only)
Tests	77 unit and integration tests, all passing
Date	18 September 2026

Student name: Arpit ChoubisaRegistration number: 24BAI10514Course: Computer Vision

1. Introduction

Surface-defect inspection on manufacturing lines is still largely performed by human operators. Attention decays over a shift, two inspectors frequently disagree on the same part, and throughput is capped by how fast a person can look. The cost of an error is asymmetric: a cracked component that reaches assembly is far more expensive to recall than one rejected at the station.

VisionInspect replaces that step with a deterministic computer-vision pipeline. It takes a grayscale surface image, decides whether the part is defective, localises every defect, and classifies its morphology as a scratch, a spot or a crack. It does this without deep learning, without a GPU and — importantly — without a stored reference image of a good part.

The defect type matters as much as the verdict. A scratch is usually cosmetic and the part can be reworked; a crack is structural and the part must be scrapped; a spot may signal a drifting process upstream. A system that answers only "good or bad" discards the information a process engineer actually needs.

Report structure

Sections 2–5 establish the problem and requirements. Sections 6–8 cover architecture, design diagrams and the reasoning behind each significant decision. Sections 9–11 describe the implementation, results and testing. Sections 12–15 cover challenges, learnings, future work and references.

2. Problem Statement

Problem. Manual visual inspection is slow, inconsistent and incomplete, and it produces a binary verdict that throws away defect-type information. An automated replacement must work without a pixel-labelled training set, without specialised hardware, and fast enough to keep pace with a production line.

Scope — included:

- Single-camera grayscale inspection of approximately planar surfaces.
- Automatic defect localisation with no manual region-of-interest setup and no golden reference image.
- Classification of each region as scratch, spot or crack.
- Per-image verdicts, batch analytics and CSV/JSON/Markdown reports.
- A reproducible synthetic dataset generator, so the project runs from a clean checkout with no downloads.
- A quantified robustness study under noise, defocus and illumination drift.

Scope — excluded: colour, 3-D, thermal or X-ray inspection; deep learning; live camera capture and PLC actuator control; sub-pixel dimensional metrology.

Target users.

User	What they need from the system
Line operator	An instant, unambiguous pass/fail verdict with the defect highlighted, so they can act without interpreting numbers.
Quality engineer	Per-defect-type counts and trends, plus a sensitivity control for when the product or lighting changes.
Process engineer	Evidence of which defect morphology is rising, to trace it back to a machine or tool upstream.

Objectives.

- Detect defects with zero false alarms on defect-free parts.
- Classify defect morphology into three named classes.

- Keep inference under 250 ms per image on a single CPU core.
- Make every stage independently testable and reproducible from a seed.
- Quantify where and how the system degrades, rather than reporting a single headline number.

3. Functional Requirements

The system is organised into three major functional modules.

Module M1 — Data and Preprocessing

ID	Requirement
FR-1.1	Generate a class-balanced synthetic dataset (good, scratch, spot, crack) from a fixed seed.
FR-1.2	Discover and validate images in a folder-per-class layout, rejecting unsupported or undecodable files with a typed error.
FR-1.3	Split the dataset into train/test partitions stratified by class.
FR-1.4	Condition every image: grayscale, resize to a fixed working resolution, denoise, equalise local contrast.

Module M2 — Detection and Classification

ID	Requirement
FR-2.1	Estimate the defect-free surface from the image itself, with no golden reference.
FR-2.2	Threshold the residual against a robust noise model and produce a cleaned binary defect mask.
FR-2.3	Extract area-filtered candidate regions ordered by area, each with a bounding box and centroid.
FR-2.4	Compute a fixed-length 13-D descriptor per region (shape, photometric, Hu moments).
FR-2.5	Train a classifier on region descriptors and persist it.
FR-2.6	Classify each region with a confidence value and derive a per-image verdict.

Module M3 — Analytics and Reporting

ID	Requirement
FR-3.1	Compute confusion matrix, accuracy and per-class precision, recall and F1.
FR-3.2	Cross-validate candidate models and hyper-parameters.
FR-3.3	Render annotated overlays and three-panel pipeline figures.
FR-3.4	Write CSV, JSON and Markdown reports including batch yield and latency.
FR-3.5	Measure accuracy under controlled image degradation.

Input / output contract

Command	Input	Output
generate	samples per class, size, seed	PNG dataset on disk
train	dataset directory	knn_model.json
inspect	one image file	verdict, findings, annotated overlay
evaluate	dataset + model	metrics, CSV/JSON/MD reports, figures

4. Non-Functional Requirements

Eight non-functional requirements were specified, each traced to the mechanism that satisfies it and the test that verifies it.

ID	Category	Requirement	How it is met	Verified by
NFR-1	Performance	Inspect an image in under 250 ms on one CPU core	Fixed 256×256 working resolution; vectorised NumPy distance computation	Measured 6.0 ms; test_latency_budget
NFR-2	Reliability	A corrupt frame must not abort a batch; no silent failures	Typed exception hierarchy; per-image try/except; config validated at construction	test_corrupt_file_is_skipped_not_fatal
NFR-3	Usability	Run the whole project with one command and no data download	python run.py pipeline; synthetic generator; --help on every sub-command	README quick start
NFR-4	Maintainability	Stages independently testable and replaceable	Layered package; each stage a pure function of (input, config); docstrings throughout	77-test suite
NFR-5	Reproducibility	Identical results across runs and machines	Every RNG seeded; deterministic split; config serialised into the report	test_preprocess_is_deterministic
NFR-6	Observability	Every stage traceable when something goes wrong	Central logging config; --log-level DEBUG; logs mirrored to outputs/logs/	logger.py
NFR-7	Resource efficiency	Run on commodity hardware with no GPU	Classical CV and k-NN; a few MB per image; no weights to download	Runs on CPU only
NFR-8	Configurability	Operators tune sensitivity without editing code	JSON config override; segment.sensitivity knob	test_from_json_applies_overrides

5. System Architecture

The system is layered, and dependencies point downward only. The domain layer knows nothing about the CLI or the file system, which is what makes each computer-vision stage unit-testable in isolation and replaceable without touching its neighbours.

- **Presentation** — `run.py` and `cli.py`: argument parsing, logging setup, console output.
- **Application** — `DefectInspector`: orchestrates the pipeline and owns the verdict rule.
- **Domain** — preprocessing, segmentation, feature extraction, classification and metrics: the actual computer vision and ML.
- **Infrastructure** — dataset generation, image loading, reporting, visualisation, configuration and logging.

System Architecture — layered view

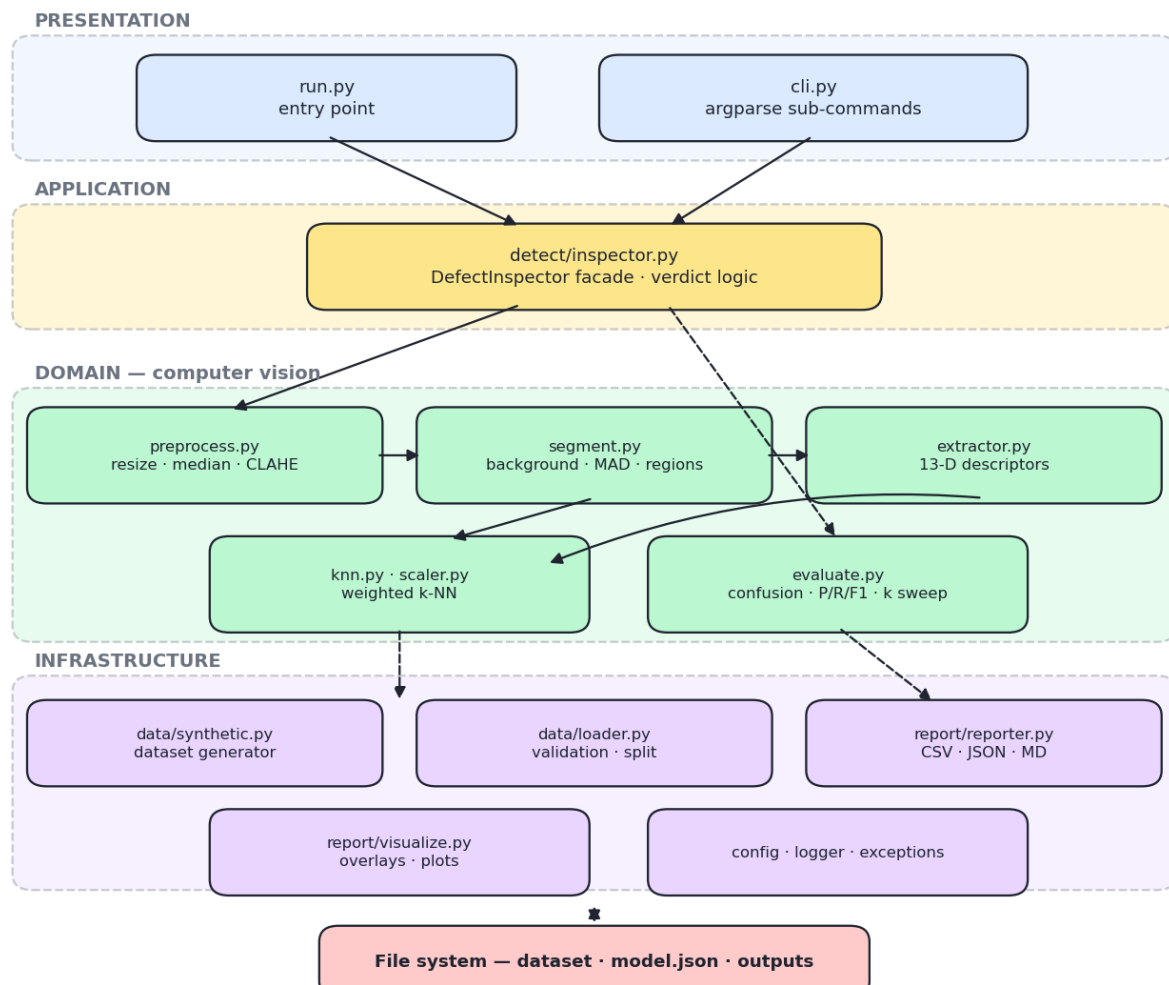


Figure 1 — Layered system architecture.

6. Design Diagrams

6.1 Process flow / workflow

The complete path a single image takes, including the branch that produces a *good* verdict when segmentation finds nothing.

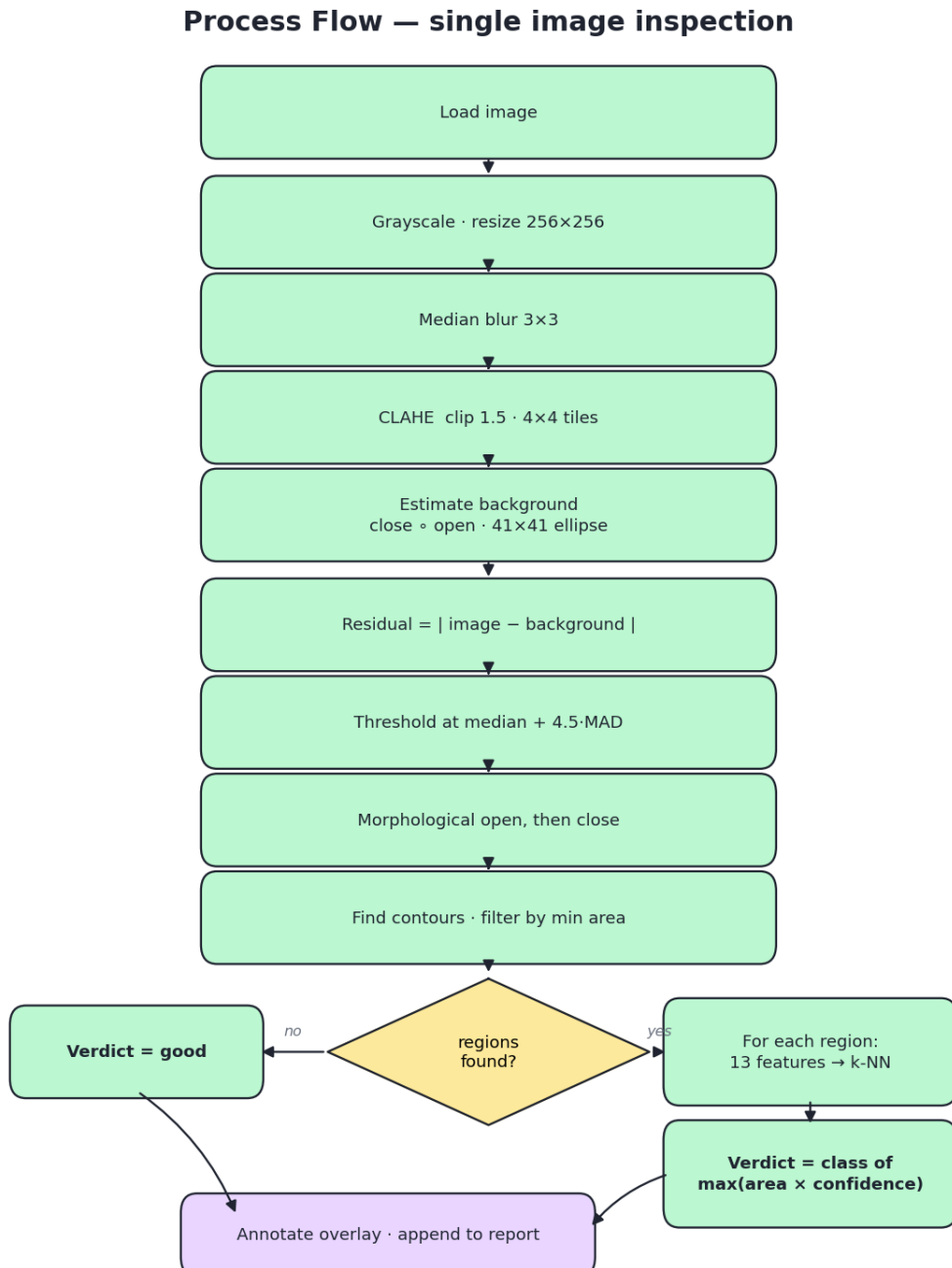


Figure 2 — Process-flow diagram for a single inspection.

6.2 Use case diagram

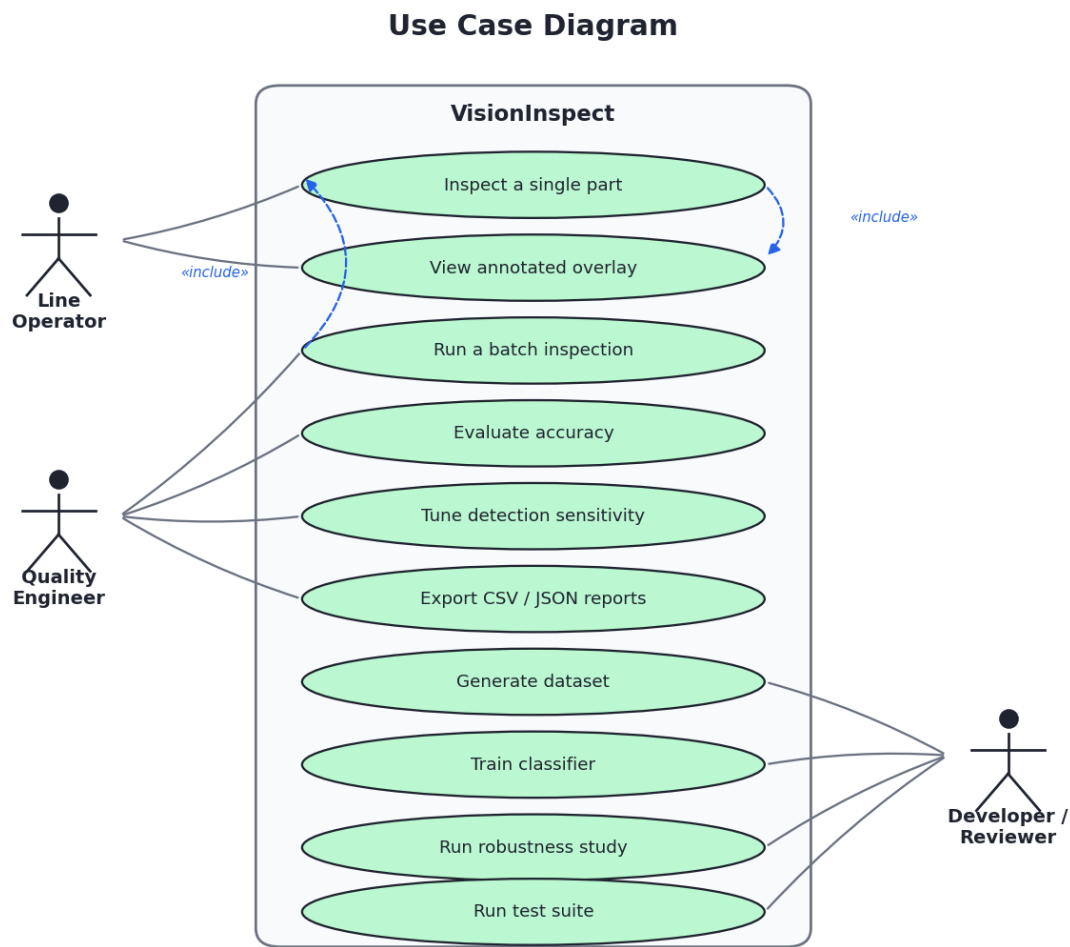


Figure 3 — Actors and the use cases they drive.

6.3 Class diagram

Class Diagram — core domain

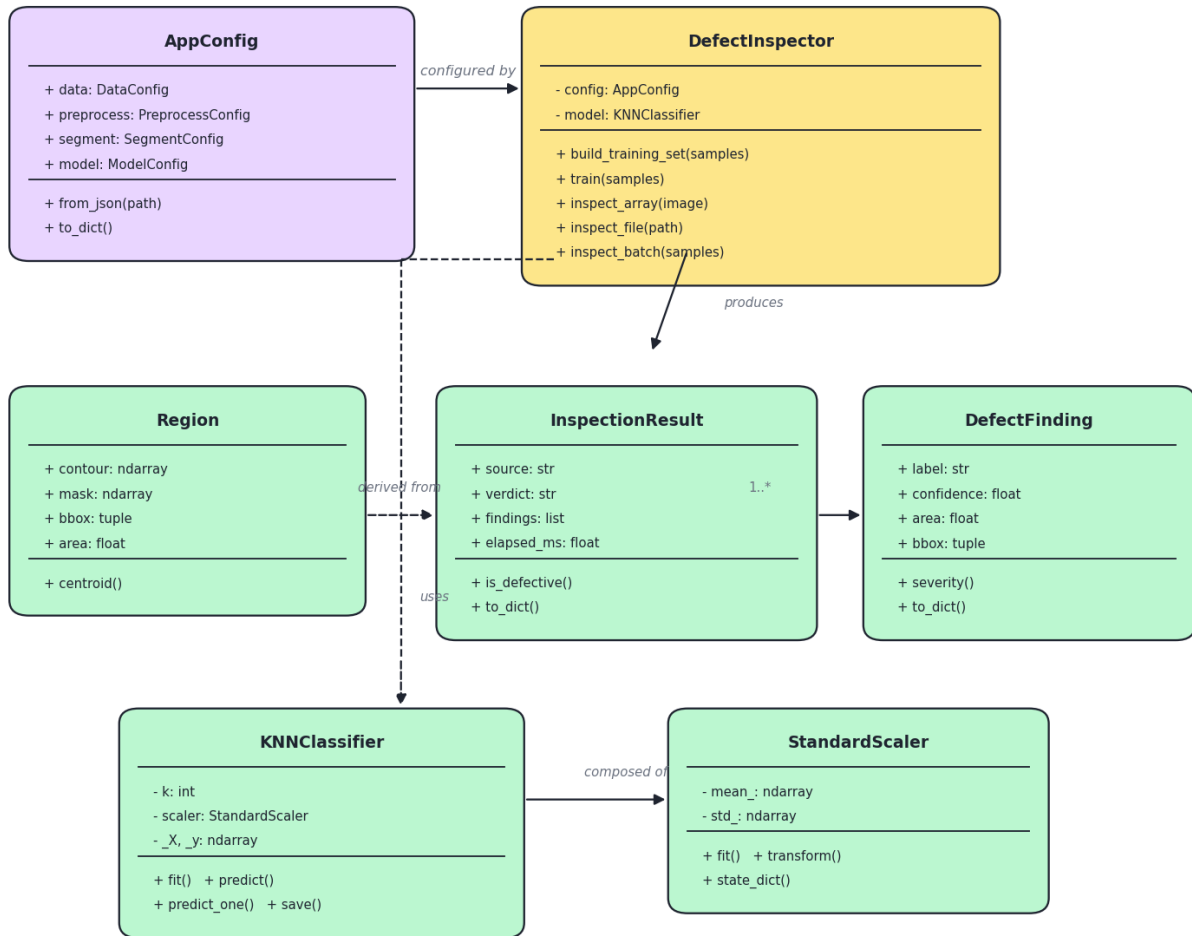


Figure 4 — Core domain classes and their relationships.

6.4 Sequence diagram

Sequence Diagram — inspect a single image

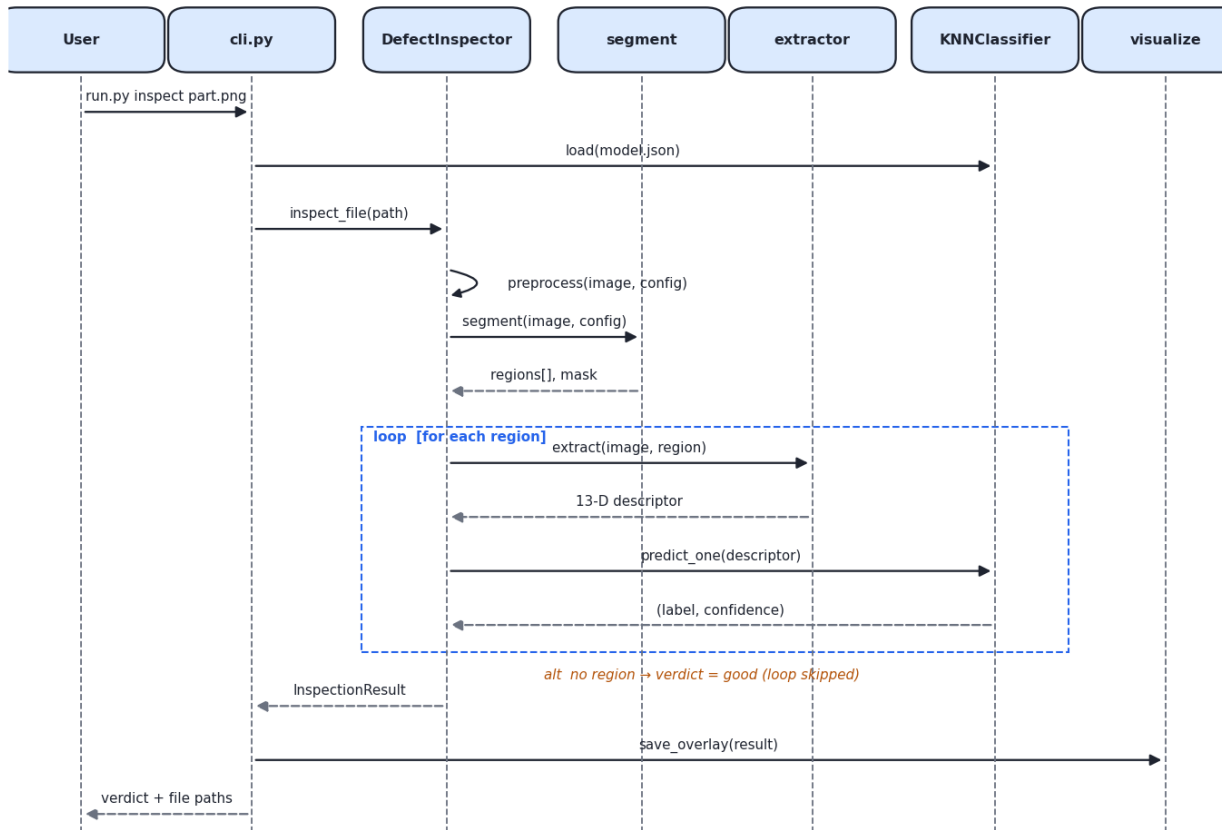


Figure 5 — Message sequence for the inspect command.

6.5 ER diagram / storage design

The project uses a structured file-system store rather than a relational database: inspection records are write-once, append-only and consumed as whole batches, so a DBMS would add a dependency without adding value. The logical entity model is nevertheless normalised, and the schema below is what a production deployment would create when persisting to SQL.

ER Diagram — inspection data model

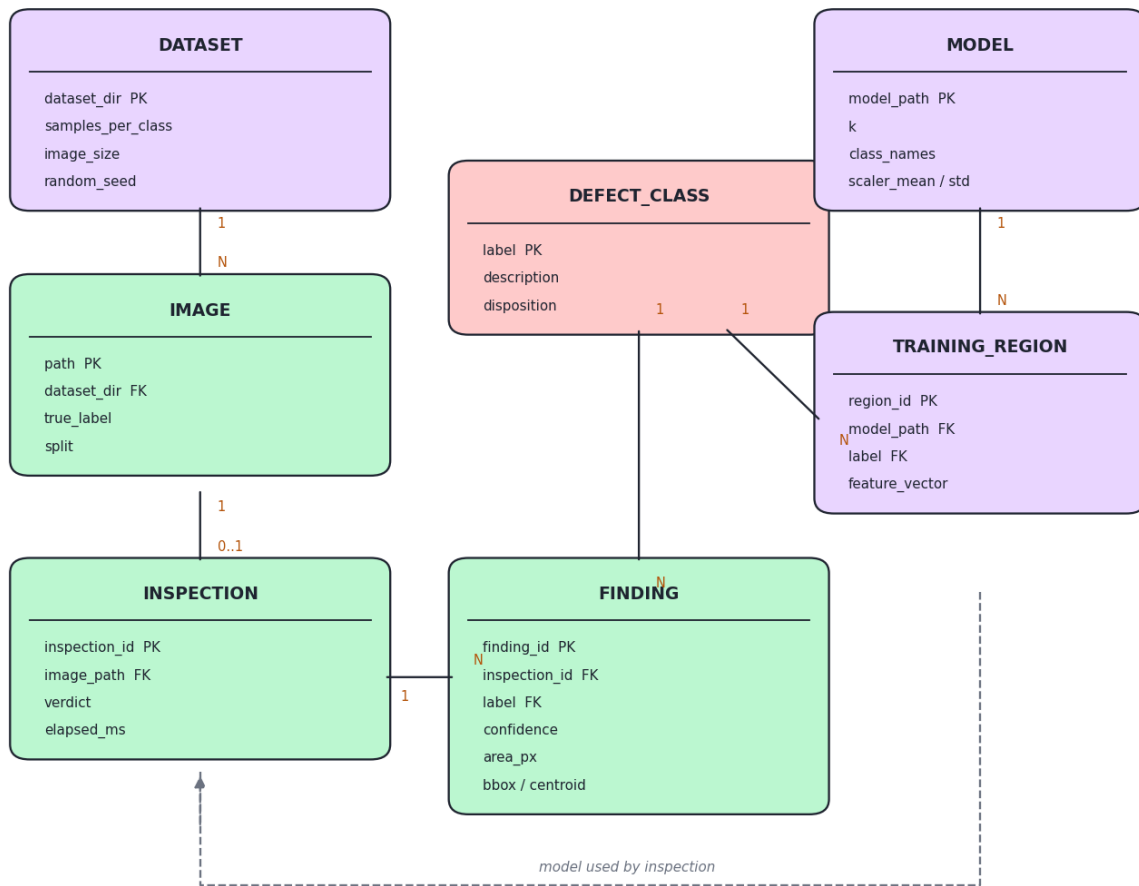


Figure 6 — Logical entity-relationship model.

Physical layout

data/dataset/<label>/<label>_NNNN.png → IMAGE
 outputs/model/knn_model.json → MODEL + TRAINING_REGION
 outputs/inspection_report.json → INSPECTION + FINDING (nested)
 outputs/inspection_report.csv → INSPECTION (flat, for spreadsheets)

7. Design Decisions and Rationale

Every significant decision, the alternatives that were considered, and the reason for the choice. Decisions D3 and D4 were driven by measured failures, described in Section 11.

#	Decision	Alternatives	Why this one
D1	Classical CV instead of a CNN	Fine-tuned ResNet, autoencoder anomaly detection	A CNN needs thousands of labelled defects and a GPU, and its failures are opaque. This pipeline trains on ~135 regions in milliseconds, and every decision traces to a nameable property.
D2	Estimate the background instead of using a golden image	Store one reference image per product and subtract	Golden-image differencing needs pixel-accurate registration and breaks whenever the product, fixture or lamp changes. Morphological reconstruction adapts per frame.
D3	MAD threshold instead of Otsu	Otsu, fixed threshold, adaptive Gaussian	Otsu always splits the histogram in two, so on a defect-free part it manufactures a defect class out of sensor noise. MAD estimates the noise floor from the median, which outlier defect pixels cannot shift. Accuracy went 35% → 93%.
D4	CLAHE clip 1.5 with 4×4 tiles	clip 2.5 with 8×8 tiles (first attempt)	8×8 tiles are 32 px wide — about a spot's diameter — so CLAHE locally normalised spots away, costing 25% of spot recall. Larger tiles with a gentler clip preserve blob contrast. 93% → 100%.
D5	Median blur, not Gaussian	Gaussian, bilateral	A 5×5 median erased 1–2 px scratches entirely. A 3×3 median removes impulse noise while keeping thin linear structures.
D6	Interpretable hand-crafted features	Raw pixels, HOG, LBP histograms	Thirteen named features make classifier errors debuggable, and three separability properties are asserted directly in the tests.
D7	Classifier written from scratch	scikit-learn KNeighborsClassifier, SVM	The course goal is understanding the algorithm. It also forced explicit choices about distance weighting and tie-breaking, and keeps the dependency list to three packages.
D8	Discard regions found in good images at training time	Train a fourth 'good region' class	Residual texture is not a defect morphology; making it a class blurs the boundary. Clean parts are handled structurally, which measured 0 false positives.
D9	Severity = area × confidence for the verdict	Largest region; highest confidence; majority vote	Area alone lets a big faint smudge outrank a small certain crack; confidence alone lets a 3-pixel speck decide.
D10	Frozen dataclass config	Dict, YAML, module constants	Invalid values fail at construction with a clear ConfigError instead of surfacing as a confusing OpenCV assertion deep in the pipeline.
D11	Synthetic dataset generator	Download NEU-DET / DAGM / MVTec-AD	Makes the repository self-contained, reproducible from a seed and licence-free — at the acknowledged cost of realism.
D12	JSON model persistence	Pickle, joblib, NPZ	Pickle is unsafe to load from an untrusted source and is version-fragile. JSON is inspectable and portable.

8. Implementation Details

8.1 Technology stack

Tool	Role
Python 3.10+	Implementation language
OpenCV ≥ 4.8	Filtering, morphology, contours, moments
NumPy ≥ 1.24	Vectorised maths, distance computation
Matplotlib ≥ 3.7	Confusion matrix, sweeps, robustness plots
ReportLab ≥ 4.0	This PDF report
unittest, argparse, dataclasses, logging, csv, json	Testing, CLI, typed config, observability, reporting
Git	Version control

8.2 Module inventory

Sixteen source modules, organised by responsibility.

Module	Responsibility
config.py	Frozen, validated dataclass configuration with JSON overrides
exceptions.py	Typed error hierarchy rooted at VisionInspectError
logger.py	Centralised logging to console and file
cli.py	argparse sub-commands: generate, train, inspect, evaluate, pipeline
data/synthetic.py	Seeded renderer for surfaces and three defect morphologies
data/loader.py	Discovery, validation, stratified splitting
pipeline/preprocess.py	Grayscale, resize, median denoise, CLAHE
pipeline/segment.py	Background estimation, MAD threshold, morphology, region extraction
features/extractor.py	13-D shape, photometric and Hu descriptors
models/scaler.py	Z-score standardisation fitted on training data
models/knn.py	Distance-weighted k-NN with JSON persistence
models/baselines.py	Nearest Centroid and Gaussian Naive Bayes
models/crossval.py	Stratified k-fold CV and model comparison
models/evaluate.py	Confusion matrix, precision/recall/F1, k sweep
detect/inspector.py	Pipeline facade and verdict logic
report/visualize.py, report/reporter.py	Overlays, plots, CSV/JSON/Markdown reports

8.3 Algorithm walkthrough

Stage	Technique	Why
1. Conditioning	Resize 256×256, median 3×3, CLAHE clip 1.5 / 4×4 tiles	Median preserves thin scratch edges a Gaussian would erase; CLAHE removes the illumination gradient so one threshold works across the frame.
2. Background estimation	Morphological close then open, 41×41 ellipse	Reconstructs the surface without the defect, removing the need for a golden reference.
3. Residual	Absolute difference	Isolates local anomalies of either polarity — a dark scratch or a bright stain.
4. Thresholding	median + 4.5 × MAD, with an absolute floor	Estimates the noise floor robustly; the floor stops the detector firing on an unusually clean frame where MAD collapses.
5. Cleanup	Open then close (3×3), area filter	Removes speckle and bridges gaps inside one defect.
6. Features	8 shape + 2 photometric + 3 Hu moments	Interpretable and rotation-invariant; separable from ~135 training regions.
7. Classification	Distance-weighted k-NN (k = 5) on z-scored features	Non-parametric, trains instantly, errors explainable by inspecting the voting neighbours.
8. Verdict	No region → good; else the class of the region with the highest area × confidence	A large confident crack outranks a small uncertain speck.

8.4 Implementation notes worth highlighting

- **Vectorised k-NN.** Distances for a whole query batch are computed with the expansion $|a-b|^2 = |a|^2 - 2a \cdot b + |b|^2$, so one matrix product replaces a Python double loop.
- **No test-set leakage.** Scaler statistics are fitted inside `fit()` on training data only, and reused unchanged at inference.
- **Numerically safe Naive Bayes.** Posteriors are computed in log space with a max-subtraction, and a variance floor prevents an infinite log-likelihood on a near-constant feature.
- **Batch resilience.** `inspect_batch` wraps each image in its own `try/except`, so one corrupt frame is logged and skipped rather than aborting a production run.

9. Dataset, Model Selection and Evaluation Methodology

9.1 Dataset description

Each image is rendered in three layers: a brushed-metal base texture (Gaussian field, $\mu = 150$, $\sigma = 6$), a radial illumination gradient of up to ± 28 grey levels from a randomly placed lamp, and zero or one defect, followed by a final $\sigma = 3$ sensor-noise pass. The illumination layer is what forces the pipeline to use adaptive rather than global thresholding.

Class	Rendering	Distinguishing property
good	No defect layer	—
scratch	Straight line, length 35–65% of frame, thickness 2–3 px, Δ intensity 55–90	High elongation, high solidity
spot	Blurred filled circle, radius 3.2–6.2% of frame, Δ intensity 62–90	High circularity, compact
crack	14–22 step random walk plus a 5–9 step branch	Elongated but low solidity from branching
Property		Value
Images		240 (60 per class, perfectly balanced)
Resolution		256 × 256, 8-bit grayscale
Train / test split		180 / 60, stratified, seed 42
Defect regions mined for training		135 (45 per defect class)
Descriptor dimensionality		13
Generation seed		7 (fully deterministic)

The classifier never sees raw pixels. The CV pipeline segments candidate regions first, and each region becomes one 13-dimensional example — converting a 65,536-pixel input into 13 interpretable numbers. That is why the model needs 135 examples rather than the thousands a CNN would require. Only the largest region per training image is labelled, and regions found in good images are discarded rather than made a fourth class (decision D8).

9.2 Model selection

Three classifiers were implemented from scratch and compared under identical stratified 5-fold cross-validation on the training split only: distance-weighted k-NN at several neighbourhood sizes, Nearest Centroid (a prototype classifier with no hyper-parameters), and Gaussian Naive Bayes (generative, assuming conditional independence). A CNN was rejected before implementation as unsuitable at this data scale; an RBF-kernel SVM was deferred.

Model	Mean accuracy	Std
k-NN (k=1)	1.0000	0.0000
k-NN (k=3, weighted)	1.0000	0.0000
k-NN (k=5, weighted)	1.0000	0.0000
k-NN (k=5, uniform)	1.0000	0.0000
k-NN (k=9, weighted)	1.0000	0.0000
Nearest Centroid	1.0000	0.0000
Gaussian Naive Bayes	0.9926	0.0148

Interpretation. Six of the seven candidates are tied at ceiling. The honest conclusion is not that k-NN is best, but that *the region-classification sub-problem is close to linearly separable in this feature space, so model choice is not the bottleneck*. Repeating the comparison on descriptors mined from images degraded with $\sigma = 18$ sensor noise produced the same tie, confirming this is a property of the feature space rather than of pristine input.

9.3 Learning curves — the experiment that does separate them

Since accuracy saturates with plenty of data, the discriminating question is how much labelled data each model needs — the realistic constraint on a new product line where few defect examples exist. Each point averages five random training subsets.

Model	n=3	n=5	n=8	n=12	n=20	n=30
k-NN (k=5)	0.986	1.000	0.998	1.000	1.000	1.000
Nearest Centroid	0.994	0.998	0.998	1.000	1.000	1.000
Gaussian Naive Bayes	0.965	0.980	0.986	0.980	0.989	0.987

Gaussian Naive Bayes plateaus near 0.98 and never closes the gap: more data does not help, because its ceiling comes from the false conditional-independence assumption rather than sample scarcity — a textbook bias-limited curve. Nearest Centroid is strongest in the extreme low-data regime, which makes sense, since estimating one mean per class is more stable than trusting individual neighbours when neighbours are scarce.

9.4 Feature ablation

Feature set	5-fold CV accuracy (k-NN, k = 5)
shape only	1.0000
photometric only	0.8132
hu_moments only	0.9680
without shape	0.9840
without photometric	1.0000
without hu_moments	1.0000
all 13 features	1.0000

Shape features carry the discriminative load, which is expected since the classes are defined by morphology. Photometric features alone reach only 0.81, confirming that how dark a defect is says much less than what shape it is. The redundancy is deliberate: the extra families cost microseconds and provide fallback signal when shape estimates degrade under blur.

9.5 Decision and evaluation protocol

Distance-weighted k-NN with k = 5 is deployed: tied for best on every accuracy measure, at ceiling from five labelled examples per class, with explainable errors and no training cost. k = 5 rather than k = 1 because, although both score 1.000 here, k = 1 has no averaging and would break first on noisier production data.

Protocol. A 75/25 stratified split with seed 42; scaler statistics fitted on training data only; model selection performed with 5-fold cross-validation *on the training split alone*, so the test split is used once for the final number. Critically, all reported metrics score the **full pipeline verdict**, not hand-picked ground-truth regions — a missed detection counts as an error, which is why the earlier 93% run showed spot recall at 0.733 rather than hiding the failure inside the detector.

10. Results

10.1 Classification performance

Held-out test split of 60 images (15 per class), never used for tuning.

Class	Precision	Recall	F1	Support
crack	1.000	1.000	1.000	15
good	1.000	1.000	1.000	15
scratch	1.000	1.000	1.000	15
spot	1.000	1.000	1.000	15
Overall	accuracy 1.000		macro-F1 1.000	60

10.2 Throughput

Metric	Value
Images inspected	60
Average latency	16.91 ms
Peak latency	51.19 ms
Throughput	59.12 images/s (single CPU thread)

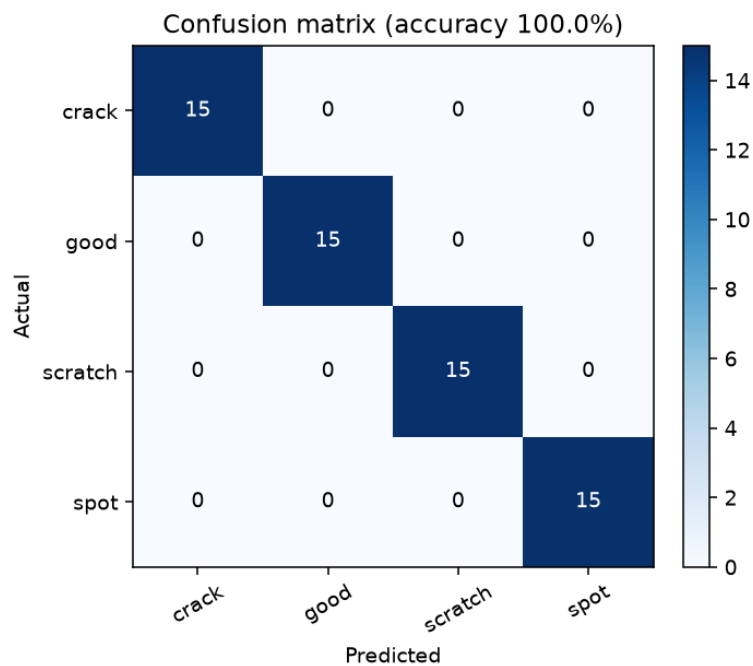


Figure 7 — Confusion matrix on the held-out test split.

10.3 Qualitative results

Each pipeline figure shows the conditioned image, the binary defect mask produced by segmentation, and the final classified overlay with bounding box, label and confidence.

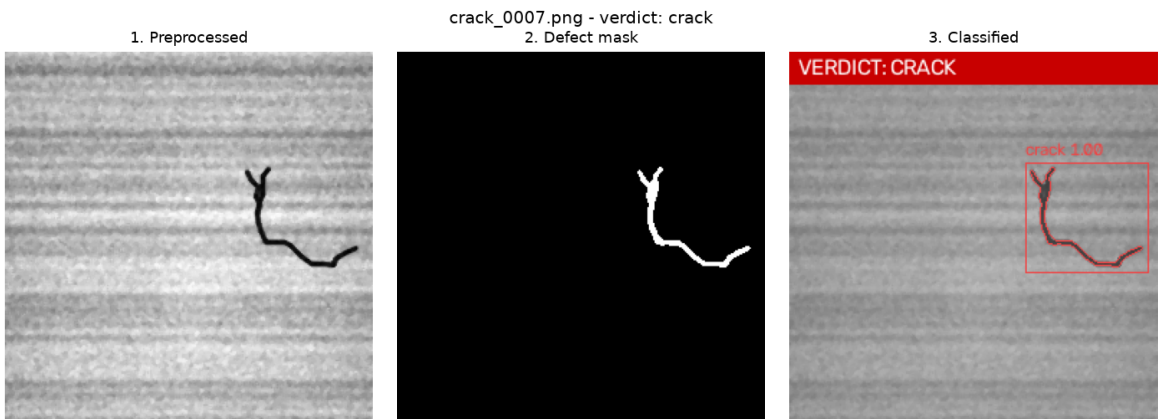


Figure 8 — Pipeline stages for crack_0007.

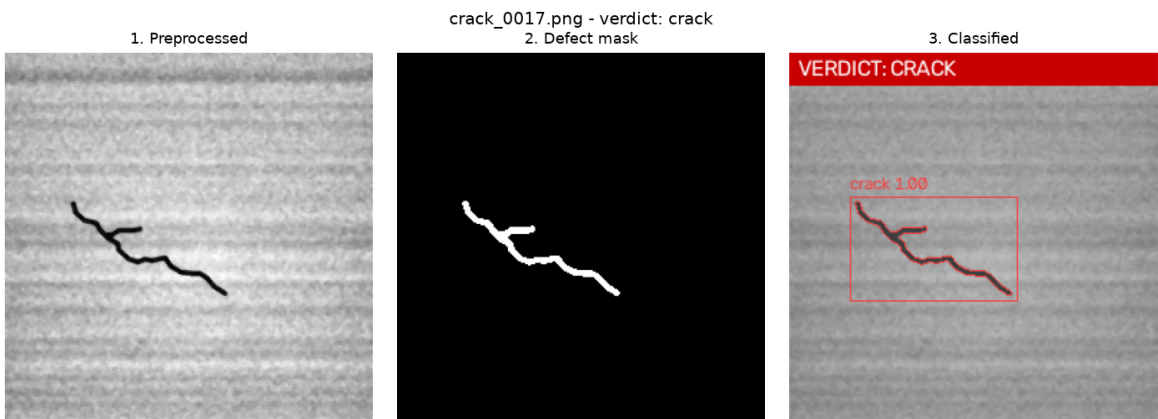


Figure 9 — Pipeline stages for crack_0017.

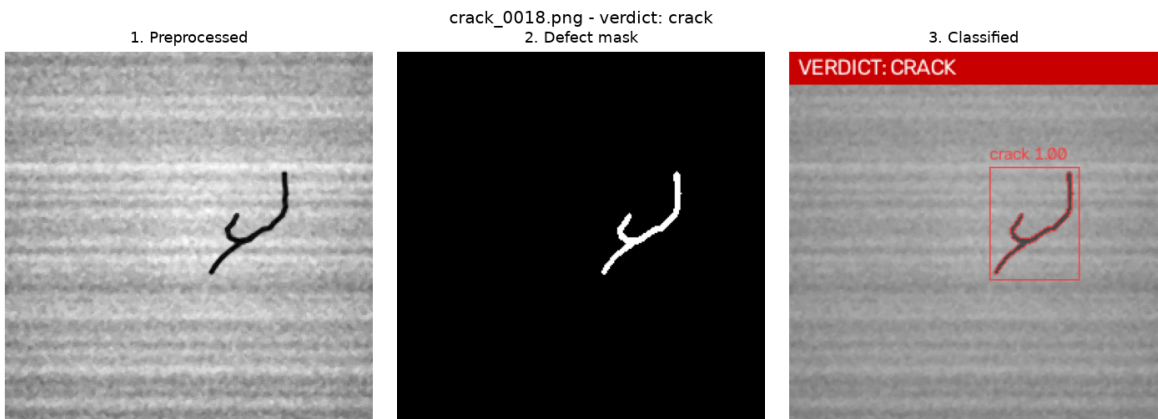


Figure 10 — Pipeline stages for crack_0018.

10.4 Model selection and hyper-parameters

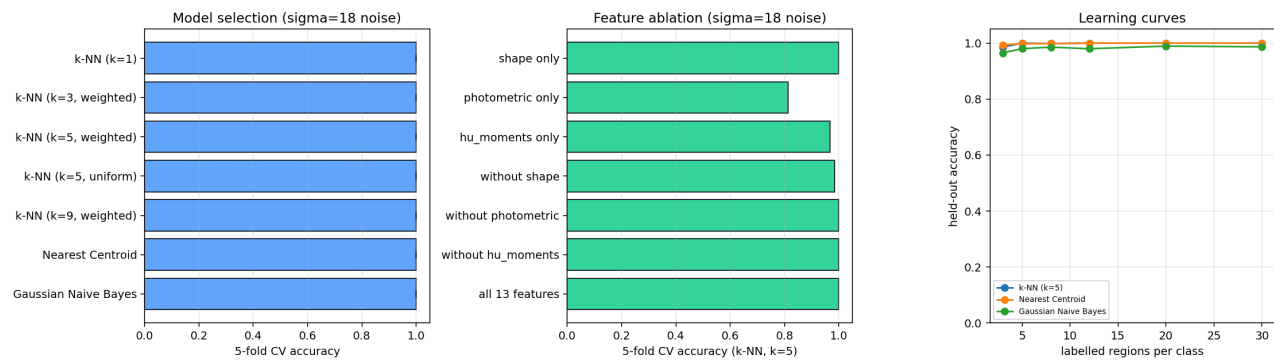


Figure 11 — Model comparison, feature ablation and learning curves.

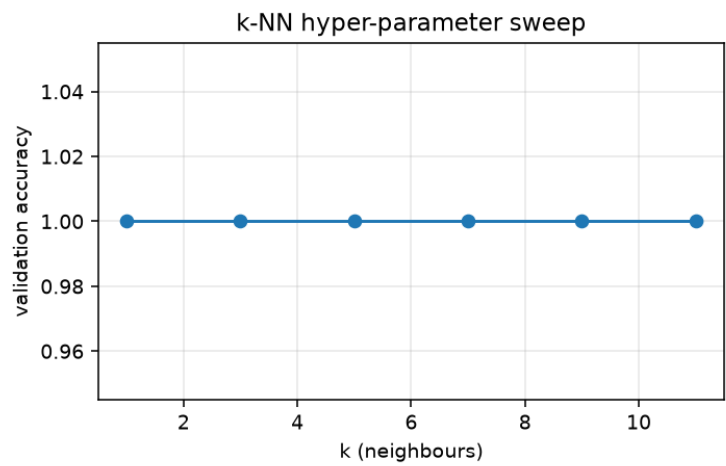


Figure 12 — Validation accuracy against neighbourhood size k.

10.5 Robustness — the number that actually matters

A perfect score on pristine synthetic renders is a weak claim on its own, so the same test split was re-scored under three controlled degradations that were never used during tuning.

Degradation	Behaviour
Gaussian sensor noise (σ)	0: 1.000, 4: 1.000, 8: 0.950, 12: 0.933, 16: 0.933, 20: 0.933, 25: 0.733
Defocus blur (kernel size)	1: 1.000, 3: 1.000, 5: 1.000, 7: 1.000, 9: 1.000, 11: 0.983
Brightness shift (grey levels)	-60: 1.000, -40: 1.000, -20: 1.000, 0: 1.000, 20: 1.000, 40: 0.950, 60: 0.933

VisionInspect robustness under image degradation

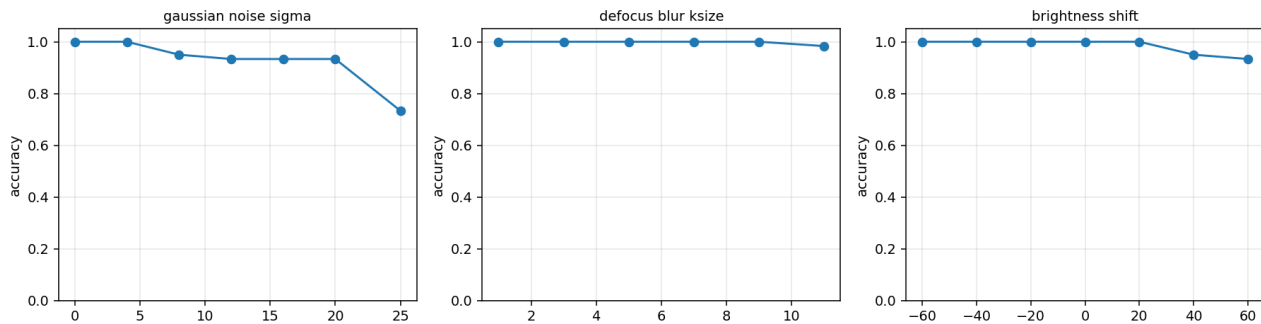


Figure 13 — Accuracy under image degradation.

The system is effectively immune to illumination drift and defocus — the morphological background estimate and CLAHE absorb both — and degrades gracefully under sensor noise. **Sensor noise is the documented failure mode:** heavy noise raises the MAD-estimated noise floor above faint defects, so they stop being detected. On a real line this argues for controlling camera gain and exposure rather than for a different classifier.

11. Testing Approach

The suite uses only the Python standard library, so no test runner needs installing: `python -m unittest discover -s tests -v` runs all 77 tests in about one second.

Test module	Covers	Tests
test_config.py	Config validation, JSON overrides, error paths	8
test_preprocess.py	Grayscale and RGBA conversion, resizing, determinism	7
test_segment.py	Zero false positives on clean parts, detection of all three defect types, mask cleanup, region ordering	8
test_features.py	Descriptor shape and finiteness, and the class-separability properties the classifier depends on	7
test_models.py	Scaler whitening, k-NN correctness, persistence round-trip, metric maths checked against hand-computed values	20
test_ml.py	Baseline classifiers, stratified fold construction, cross-validation and model comparison	14
test_integration.py	End-to-end accuracy, latency budget, corrupt-file resilience, report writing	13

Testing philosophy

- **Test the property, not the implementation.** Feature tests assert that scratches are more elongated than spots and that cracks are less solid than spots — the assumptions the classifier relies on. If a future refactor breaks those, the tests catch it.
- **Guard the expensive failure.** A dedicated test asserts zero regions on five different clean surfaces, because a false alarm on a good part is the failure that first destroyed the system's accuracy.
- **Verify the non-functional requirements too.** Latency, determinism and corrupt-file resilience each have an explicit test, so the NFR table in Section 4 is backed by executable checks.
- **Hand-computed expectations for metric code.** Precision and recall are checked against values computed by hand, since an incorrect metric would silently invalidate every reported result.

12. Challenges Faced

12.1 Otsu thresholding invented defects on clean parts

Symptom. The first working version scored 35% accuracy, and the *good* class had recall 0.000 — every defect-free part was flagged, typically with eight phantom regions.

Diagnosis. Printing residual statistics per class showed the cause: on a clean part the residual contains only sensor noise, but Otsu's method always partitions a histogram into two classes, so it obediently split the noise and labelled the upper half 'defect'. The algorithm was working exactly as designed; the design was wrong for this problem.

Fix. Replace Otsu with a robust noise model: estimate the floor as median + $4.5 \times \text{MAD}$, which the small fraction of outlier pixels a real defect contributes cannot shift, and add an absolute floor for unusually clean frames. False positives dropped to zero and accuracy rose to 93%.

12.2 CLAHE was erasing the defects it was meant to reveal

Symptom. After the threshold fix, spot recall sat at 0.733 while scratches and cracks were perfect — a class-specific failure, which pointed at something size-dependent rather than a general threshold problem.

Diagnosis. Contrast-limited adaptive histogram equalisation with an 8×8 tile grid divides a 256 px image into 32 px tiles. A spot is 16–32 px across, so it occupied most of its own tile — and CLAHE, doing its job, normalised the spot's contrast away before the detector ever saw it. Scratches and cracks span many tiles and survived.

Fix. A sweep over tile size and clip limit showed that 4×4 tiles with clip 1.5 preserve blob contrast while still removing the illumination gradient: spot detection went from 45/60 to 60/60, and end-to-end accuracy from 93% to 100%.

12.3 Median blur was deleting thin scratches

A 5×5 median filter removes any structure narrower than about half the kernel — which is exactly what a 1–2 px scratch is. Reducing the kernel to 3×3 and rendering scratches at a realistic 2–3 px recovered scratch detection to 60/60.

12.4 A perfect score is a reporting problem

Reaching 100% created an unexpected difficulty: the result is impossible to interpret. Six of seven candidate classifiers tie at ceiling, so cross-validation cannot rank them, and the number says nothing about production behaviour. The response was to add experiments that *can* discriminate — a robustness study under noise, blur and illumination drift, learning curves at small sample sizes, and a feature ablation — and to state the synthetic-data limitation plainly rather than presenting 100% as a production claim.

12.5 Test tooling was unavailable offline

The suite was originally written for pytest, which could not be installed in the target environment. Rather than ship tests that would not run for a reviewer, the suite was converted to the standard library's unittest — a useful reminder that a dependency is a liability when reproducibility is the point.

13. Learnings and Key Takeaways

- **A textbook algorithm can be exactly wrong for a problem.** Otsu is the standard answer to automatic thresholding, and it was the single worst decision in the project. Its assumption — that the image contains two populations — is false for a defect-free part. Matching an algorithm's assumptions to the data matters more than its reputation.
- **Preprocessing can destroy the signal it is meant to enhance.** CLAHE removed exactly the defects whose size matched its tile grid. Every preprocessing step has a scale at which it becomes destructive, and that scale must be checked against the size of the feature of interest.
- **Measure the system, not the component.** Scoring the end-to-end verdict rather than hand-picked regions made a detection failure visible as spot recall of 0.733. Component-level metrics would have shown a perfect classifier and hidden the real bug.
- **Class-specific failures are the most informative.** When one class failed while two succeeded, the error pattern pointed directly at a size-dependent cause and made the CLAHE diagnosis quick.
- **Pick the simplest model that works, then prove it.** k-NN, Nearest Centroid and Naive Bayes all reach ceiling; the justification for k-NN comes from learning curves and explainability, not from a difference in headline accuracy.
- **Robustness is more honest than accuracy.** The most useful number in this report is not 100% — it is that accuracy falls to 0.733 at $\sigma = 25$ noise, because that identifies the failure mode to engineer against.
- **Interpretable features pay for themselves.** Because features are named quantities, the ablation is meaningful, the tests assert real properties, and classifier errors can be explained by printing the neighbours that voted.

14. Future Enhancements

- **Validate on real data.** Benchmark against NEU-DET to quantify the synthetic-to-real gap — the single most valuable next step.
- **Add an SVM and a small CNN** once a real dataset makes the comparison meaningful and the ceiling effect disappears.
- **Replace linear search with a KD-tree** so inference cost stops scaling with training-set size.
- **Publish a precision–recall curve** by sweeping segment.sensitivity, giving operators an explicit escape-rate versus false-alarm trade-off.
- **Report every finding** rather than a single dominant class, so multi-defect parts are described fully.
- **Wrap the inspector in a REST service** and stream frames from a live camera, with a reject-actuator interface.
- **Add statistical process control** — track defect-rate trends and raise a control-chart alarm when one class rises, which is the feature a process engineer would use daily.
- **Nested cross-validation**, so hyper-parameter selection and performance estimation are fully separated.

15. References

- Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1), 62–66.
- Hu, M. K. (1962). Visual pattern recognition by moment invariants. *IRE Transactions on Information Theory*, 8(2), 179–187.
- Zuiderveld, K. (1994). Contrast limited adaptive histogram equalization. In *Graphics Gems IV*, Academic Press, 474–485.
- Serra, J. (1982). *Image Analysis and Mathematical Morphology*. Academic Press.
- Suzuki, S. and Abe, K. (1985). Topological structural analysis of digitized binary images by border following. *Computer Vision, Graphics, and Image Processing*, 30(1), 32–46.
- Rousseeuw, P. J. and Croux, C. (1993). Alternatives to the median absolute deviation. *Journal of the American Statistical Association*, 88(424), 1273–1283.
- Cover, T. and Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
- Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*. Documentation: <https://docs.opencv.org>
- Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- Song, K. and Yan, Y. (2013). A noise-robust method based on completed local binary patterns for hot-rolled steel strip surface defects. *Applied Surface Science*, 285, 858–864. (NEU-DET dataset, referenced as future validation data.)